Knowledge Representation and Reasoning in AI

Source: NPTEL course on Fundamentals of Artificial Intelligence, by Prof. Shyamanta M. Hazarika

Knowledge Representation and Reasoning

- As we always say in AI, we have to make the agents and machines intelligent.
- To make the machines intelligent, one of the major key factor is knowledge.

Knowledge Representation and Reasoning

- How does a human being accumulate the knowledge?
- How to define knowledge?
  - Knowledge is facts and skills, which we accumulate through the education and experience.
  - And the intelligence is the ability to use that knowledge.
    - We use that knowledge at the right time.
- Main important thing is the reasoning.
  - Reasoning is the processing of the knowledge (i.e. thinking)

Knowledge Representation and Reasoning

- **Example**: we know that if you jump off the 25th floor, noting will be left. This is the knowledge.
- Reasoning?
  - Based on the reasoning, you will reach to a valid conclusion that whether you should jump or not.
  - If you jump, it means that you didn't show intelligence.
  - If you don't jump, you showed your intelligence.

Knowledge Representation and Reasoning

- **key main factors: knowledge, reasoning, and intelligence**. We want to put all three in machines because the machine does not educate itself.
  - Machine is not experiencing itself. It will make decisions based on what we will provide it.

Knowledge representation

- Knowledge representation is the key factor.
  - Why?
    - Knowledge representation means that we have to store the knowledge so that machine can understand the knowledge at the right time and make a proper decision.
  - If we represent the wrong knowledge, then obviously, machine will interpret incorrectly.

Knowledge representation (contd…)

- If machine will interpret incorrectly, the result will also be incorrect.
- So we should more focus on the knowledge representation.
- **Example**: The program that you will write, the output will be according to that. If you write and represent the program in the correct manner, then obviously, your result will also be correct.

Knowledge representation (contd…)

- If there is no proper knowledge representation, then there can be error…
  - **Syntax error**:
  - **Semantic error**:
  - If you make minimum syntax and semantic errors, then your representation will be correct and further understanding will also be correct.

Knowledge representation methods

- **Logic**
  - **Propositional logic-** it is used in terms of true OR false
  - **Predicate logic-** we use quantifiers like for all, and there exist. We use different types of predicate logic.
- **Rules**- we use if-then conditions as rules and on the basis of that, the machine acts.
- **Semantic Net** (example: Google graph)- A big graph that will have relationships, and objects and those objects will be interrelated.
- **Frame**- slots (object) and fillers (attributes)
- **Script** (advance version of frame)- for example movie has action, songs, different roles. We will give that whole script to the agent/machine.

Propositional Logic

Propositional Logic

- Propositional Logic (Output should be Either True or False, not both)
  - Example:
    - 1+1=2 i.e. True Proposition ; 2+1=4 i.e. false proposition
    - New Delhi is the capital of India i.e. True
    - Some students are intelligent - T/F i.e. is not the part of propositional logic
- Types of proposition
  - Atomic (single proposition i.e. sky is blue)- there is no use of symbols
  - Complex OR Compound Propositions (where two or more than two sentences together forming a one proposition) – set of symbols help in forming propositions

Proposition

Definition: A **proposition** is a statement that is, by itself, either true or false.

Sample Propositions

Can be either true or false.

- All humans are mortal.
- Ram is married.
- I'll pay for the meal.

Things that aren't propositions

Cannot be true or false.

- Come here! Command.
- Why are you crying? Question.

Proposition Logic

Propositional logic is a **mathematical system** for **reasoning about propositions** and **how they relate to one another.**

- Every statement in propositional logic consists of propositional variables combined via propositional connectives.
- Each variable represents some proposition.

  It is hot.
  It is humid.

- Connectives encode how propositions are related.

  If it is humid, then it is hot

Propositional Variable

Definition: A **propositional variable** represents an arbitrary proposition. We represent propositional variables with uppercase letters.

P It is hot.

Q It is humid.

Definition: Each variable can take one of two values: true or false. If a proposition is true, then we say its **truth value** is true, and if a proposition is false, we say its truth value is false.

Propositional Connectives

Logical NOT: ¬ P

- Read "not P"
- ¬ P is true if and only if P is false.
- Also called **logical negation**.

Logical AND: P ∧ Q

- Read "P and Q."
- P ∧ Q is true if both P and Q are true.
- Also called **logical conjunction**.

Logical OR: P ∨ Q

- Read "P or Q."
- P ∨ Q is true if at least one of P or Q are true.
- Also called **logical disjunction**.

The OR operator is an *inclusive* OR; It is true if at least one of the operands is true.

Propositional Connectives

Implication: $P \rightarrow Q$

- Read "If P then Q".
- False when P is true and Q is false; and is true otherwise.
- Also called **material conditional operator**.

Biconditional: $P \leftrightarrow Q$

- Read "P if and only if Q".
- True if P and Q have the same truth values; and false otherwise.
- Also called **material biconditional operator**

true and **false**:

- The symbol $\top$ is a value that is always true.
- The symbol $\bot$ is a value that is always false.

Well-formed Formula

Definition: A **sentence** also called a **well-formed formula** is defined as follows:

- A symbol S is a sentence
- If S is a sentence, then $\neg$S is a sentence
- If S is a sentence, then (S) is a sentence
- If S and T are sentences, then

  i. $(S \vee T)$ ii. $(S \wedge T)$ iii. $(S \rightarrow T)$ and iv. $(S \leftrightarrow T)$ are sentences
- A sentence results from a finite number of applications of the above rules.

Truth Table

A **truth table** is a table showing the truth value of a propositional logic formula as a function of its inputs.

- Represent the relationship between the truth values of propositions and compound propositions formed from those propositions.

P	Q	P ∨ Q
T	T	T
T	F	T
F	T	T
F	F	F

Truth Table

A **truth table** is a table showing the truth value of a propositional logic formula as a function of its inputs.

- Represent the relationship between the truth values of propositions and compound propositions formed from those propositions.
- Formally defining what a connective `means'.
- Deciphering complex propositional formula.

Implication

For propositions **P** and **Q**, **P → Q** , the **implication** or **conditional statement** is **false** when **P is true** and **Q is false**, and is **true otherwise**.

- P is called the premise or hypothesis.
- Q is called the conclusion.

P	Q	P → Q
F	F	T
F	T	T
T	F	F
T	T	T

We want P → Q to mean
"whenever P is true, Q is true as well."

Only way this doesn't happen is if P is true and Q is false.

Implication

In English, a sentence of the form `if A then B' can have different meanings.

1. Typically there is a causal relationship between A and B, which is not required in logic.
2. We are often implying more than simply that if A holds, then B holds as well.

- Example

  If I earn a bonus, then I will buy a car.

  P: I earn a bonus.

  Q: I will buy a car.

  $P \rightarrow Q$

  - The common-sense interpretation of this sentence is that the inverse statement is also true:

    If I *do not* earn a bonus, then I will *not* buy a car.

  This is not implied by $P \rightarrow Q$.

Biconditional

The **biconditional** of statements **P** and **Q**, denoted **P ↔ Q** , is read "P if and only if Q", and is **true** if **P** and **Q** have the **same truth values**, and **false otherwise**.

The biconditional operator is used to represent a two-directional implication.

P	Q	P ↔ Q
F	F	T
F	T	F
T	F	F
T	T	T

Specifically, p ↔ q means that p implies q and q implies p.

Conversely if both P implies Q and Q implies P are true, then P if and only if Q is true.

Operator Precedence

Operator precedence for propositional logic:

¬

∧

∨

→

↔

NOT binds to whatever immediately follows it.

∧ and ∨ bind more tightly than →

- All operators are right-associative.
- Parentheses can be used to disambiguate.

Operator precedence

$$\neg x \rightarrow x \vee z \wedge y$$

$$(\neg x) \rightarrow (x \vee (z \wedge y))$$

Translating English Into Logic

User defines a **set of propositional symbols**, like P and Q.

User defines the **semantics** of each **propositional symbol**:

P	It is hot.
Q	It is humid.
R	It is raining.

1. If it is humid, then it is hot
   $Q \rightarrow P$

2. If it is hot and humid, then it is raining.
   $(P \wedge Q) \rightarrow R$

Translating English Into Logic

W I will work hard.
V There are vacancies.
J I will get the job.

If I don't work hard, then I won't get the job.

3. I won't get the job, if I don't work hard.

$$\neg W \rightarrow \neg J$$

P if Q

translates to

$$Q \rightarrow P$$

Translating English Into Logic

W I will work hard.
V There are vacancies.
J I will get the job.

4. If I work hard but there are no vacancies, I won't get the job.

Because the second part of the sentence is a surprise, "but" is used instead of "and".

$$(W \wedge \neg V) \rightarrow \neg J$$

P, but Q

translates to

$P \wedge Q$

De Morgan's Laws

Using truth table, we conclude

$\neg(P \wedge Q) \equiv (\neg P \vee \neg Q)$

P	Q	$\neg(P \wedge Q)$	$\neg P \vee \neg Q$
T	T	F	F
T	F	T	T
F	T	T	T
F	F	T	T

De Morgan's Laws

Using truth table, we conclude

$\neg(P \vee Q) \equiv (\neg P \wedge \neg Q)$

P	Q	$\neg(P \vee Q)$	$\neg P \wedge \neg Q$
T	T	F	F
T	F	F	F
F	T	F	F
F	F	T	T

Logical Equivalence

$\neg(P \wedge Q)$ and $(\neg P \vee \neg Q)$ have the same truth tables, we say that they're **equivalent** to one another.

$\neg(P \wedge Q) \equiv (\neg P \vee \neg Q)$

- The $\equiv$ symbol is not a connective. It's related to $\leftrightarrow$, but it's not the same:
- The statement $\neg(P \wedge Q) \equiv (\neg P \vee \neg Q)$ means `the two formulas are equivalent.'

The formula evaluates to true every time.

- The statement $\neg(P \wedge Q) \leftrightarrow (\neg P \vee \neg Q)$ is a propositional formula. If you plug in different values of P and Q, it will evaluate to a truth value.

Logical Equivalence

Here's a useful equivalence.

$P \rightarrow Q \equiv (\neg P \vee Q)$

P	Q	$P \rightarrow Q$	$\neg(P \wedge \neg Q)$
T	T	T	T
T	F	F	F
F	T	T	T
F	F	T	T

Start with $P \rightarrow Q \equiv \neg (P \wedge \neg Q)$

By De Morgan's laws:

- $P \rightarrow Q \equiv \neg P \vee \neg\neg Q$
- $P \rightarrow Q \equiv \neg P \vee Q$

Thus $P \rightarrow Q \equiv \neg P \vee Q$

Rules of Inference

A **rule of inference** is sound if its conclusion is true whenever the premise is true.

- Here are some examples of sound rules of inference.
- Each can be shown to be sound using a truth table.

RULE	PREMISE	CONCLUSION
Modus Ponens	$A, A \rightarrow B$	B
And Introduction	A, B	$A \wedge B$
And Elimination	$A \wedge B$	A
Double Negation	$\neg \neg A$	A
Unit Resolution	$A \vee B, \neg B$	A
Resolution	$A \vee B, \neg B \vee C$	$A \vee C$

Proving Theorems

A **proof** is a **sequence of sentences**, where each sentence is either a premise or a sentence **derived from earlier sentences** in the proof by one of the rules of inference.

- The last sentence is the **theorem** (also called goal or query) that we want to prove.

- Example

1. Q	Premise	It is humid
2. Q→P	Premise	If it is humid, it is hot
3. P	Modus Ponens(1,2)	It is hot
4. (P∧Q)→R	Premise	If it's hot and humid, it's raining
5. P ∧ Q	And Introduction(1,3)	It is hot and humid
6. R	Modus Ponens(4,5)	It is raining

From 1 and 2, I could use modus ponens to arrive at that it is hot Q implies P, Q therefore P so I can say it is hot.

So, here in this example I have the propositional formulas in red as my premise and in black the sentence that has been derived either from earlier sentences or from the premise itself. And finally I could have a theorem that it is raining.

Propositional Logic

Commits only to the existence of facts that may not be the case in the world being represented.

- **Logical constants**: true, false
- **Propositional symbols**: P, Q, S, ... (atomic sentences)
- Wrapping **parentheses**: (…)
- Sentences are combined by **propositional connectives**:

$\wedge$	and	[conjunction]
$\vee$	or	[disjunction]
$\rightarrow$	implies	[implication / conditional]
$\leftrightarrow$	is equivalent	[biconditional]
$\neg$	not	[negation]

It has a simple syntax and simple semantics. It suffices to illustrate the process of inference. Propositional logic quickly becomes impractical, even for very small worlds.

Weak Language

Propositional Logic is a **weak Language.**

- Consider the problem of representing the following information:
  - Every person is mortal.
  - Socrates is a person.
  - Socrates is mortal.

Although the third sentence is entailed by the first two, an explicit symbol, to represent an individual was required.

- How can these sentences be represented so that we can infer the third sentence from the first two?
  - Create propositional symbols.

    P = He is a Person; M = He is Mortal; S = Socrates
  - $P \rightarrow M$; $S \rightarrow P$; Therefore $S \rightarrow M$

To represent other individuals we need separate symbols for each one; some way to represent the fact that all individuals who are "people" are also "mortal".

First-Order Logic

- Propositional Logic
  - Hard to identify "individuals"
    - E.g., Mary, 3
  - Can't directly talk about properties of individuals or relations between individuals
    - E.g., Ben is fat.
  - Generalizations, patterns, regularities can't easily be represented
    - E.g., All triangles have 3 sides.
- First-Order Logic
  - FOL or FOPC is expressive enough to concisely represent this kind of information
  - FOL adds relations, variables, and quantifiers, e.g.,
    - Every elephant is gray. : $\forall$ x (elephant(x) $\rightarrow$ gray(x))
    - There is a white alligator.: $\exists$ x (alligator(X) ^ white(X))

First-order logic allows us to get at the internal structure of certain propositions in a way that is not possible with propositional logic.

First Order Logic

Language to formulate Knowledge

- A system aspiring to be intelligent, need to be able to formulate knowledge of the world!

  Propositional Logic is a weak Language!

  - Language of our choice is the First-order Logic
    - Simple and Convenient to begin with.
- Three things of a *language* that are of our concern
  - **Syntax**
    - Specify which group of symbols, arranged in what way, are to be considered properly formed.
  - **Semantics**

    In English - There is someone behind you; Warning! Or Request
    - Specify what the well-formed expressions are supposed to mean.
  - **Pragmatics**

    In KR &R - How to use meaningful sentences as part of a KB from which inferences are drawn.
    - Specify how the meaningful expression are to be used.

First-Order Logic

- First-order logic is a logical system for reasoning about properties of objects.
- Augments the logical connectives from propositional logic. and adds
  - **predicates** that describe properties of objects.
  - **functions** that map objects to one another.
  - **quantifiers** to reason about multiple objects simultaneously.

First-Order Logic

- First-order logic models the world in terms of
  - **Objects**
    - Things with individual identities
  - **Properties**
    - Distinguish objects from other objects.
  - **Relations**
    - Hold among sets of objects.
  - **Functions**
    - subset of Relations; one value for a given input.

The notion of an *object* is quite broad. Objects can be concrete or abstract; Objects can be primitive or composite.

A relation takes objects as arguments and generates a truth value. Functions applied to arguments name things.

Propositional Logic

- Each variable represents a proposition, which is either true or false.
- Directly apply connectives to propositions:
  - $\neg P \wedge Q$
  - $P \rightarrow Q$
- The truth of a statement can be determined from the truth values for the input propositions.
  - Can see all possible truth values for a statement by checking all possible truth assignments to its variables.

First-Order Logic

- Each variable refers to some object in a set called the **domain of discourse**.
- First-order variables refer to arbitrary objects, it does not make sense to directly apply connectives to them:
- To reason about objects, first-order logic uses **predicates.**
  - In English, the predicate is the part of the sentence that tells you something about the subject.

Predicate

Definition: A **predicate** is a property that a variable or a finite collection of variables can have.

- Predicates can take any number of arguments, but each predicate has a fixed number of arguments called its arity.
  - $P(x_1, x_2, .., x_n)$ is a predicate of n variables or n arguments.
- A predicate becomes a proposition when specific values are assigned to the variables.
  - Applying a predicate to arguments produces a proposition, which is either true or false.

Predicate

- Example
  - She is a student at IIT Guwahati.

    We could have a predicate

    P(x, IIT) - `x' is a student at IIT Guwahati.

    OR

    P(x, y) - `x' is a student at `y'.
  - He lives in the city.

    We could have a predicate

    P(x, y) - `x' lives in `y'.

    Note that P(Mohan, Guwahati) is a proposition!

Functions in FOL

Definition: A **function** return objects associated with other objects.

- Functions can take any number of arguments, but each function has a fixed number of arguments called its arity.
  - $F(x_1, x_2, ..., x_n)$ is a function of n variables or n arguments.
- Functions evaluate to objects, not propositions when specific values are assigned to the variables.
  - MotherOf(x): a function that returns the mother of `x'.
    MotherOf(Jesus) would return `Mary'.

Syntax of First-Order Logic

Two types of symbols

- Variables
  - A variable is any sequence of *lowercase* alphabet and numeric characters in which the first character is lowercase alphabet.
- Constants
  - Object Constants
    - An object constant is used to name a specific element of a universe of discourse.
  - Function Constants
    - A function constant is used to designate a function on members of the universe of discourse.
  - Relation Constants
    - A relation constant is used to name a relation on the universe of discourse.

Syntax of First-Order Logic

FOL Provides

- Variable symbols
  - E.g., x, y, foo
- Connectives
  - Same as in PL:
  - $\neg$, $\wedge$, $\vee$, $\rightarrow$, $\leftrightarrow$
- Quantifiers
  - Universal $\forall x$
  - Existential $\exists x$

A predicate becomes a proposition when specific values are assigned to the variables

User Provides

- Constant symbols
  - Mary
  - Green
- Function symbols
  - father-of(Mary) = John
  - color-of(Sky) = Blue
- Predicate symbols
  - greater(5,3)
  - color(Grass, Green)

Functions evaluate to objects, not propositions when specific values are assigned to the variables

Syntax of First-Order Logic

- A **term** (denoting a real-world individual) is a constant symbol, a variable symbol, or an n-place function of n terms.

  x and $f(x_1, ..., x_n)$ are terms, where each x_i is a term.

  A term with no variables is a **ground term.**

  In FOL, facts are stated in the form of expressions called sentences or well-formed formulas.

- An **atomic sentence** (which has value true or false) is an n-place predicate of n terms.

Syntax of First-Order Logic

- A **complex sentence** is formed from atomic sentences connected by the logical connectives:

  $\neg P$, $P \vee Q$, $P \wedge Q$, $P \rightarrow Q$, $P \leftrightarrow Q$ where P and Q are sentences

- A **quantified sentence** adds quantifiers $\forall$ and $\exists$
  - Universally quantified
  - Existentially quantified

  Quantified sentences provide a more flexible way of talking about objects in the universe of discourse.

- A **well-formed formula** (wff) is a sentence containing no "free" variables. That is, all variables are "bound" by universal or existential quantifiers.

Equality

- First-order logic includes a special predicate =
  - States whether two objects are equal to one another.
  - Example
    - Two = 2
  - **Equality** symbol (=) is a logical constant and can be best understood as the identity relation.
- Equality can only be applied to object.
  - Biconditional ↔ is used to see if propositions are equal.
- Define ≠ as $x \neq y \equiv \neg(x = y)$

Equality is a part of first-order logic

First Order Logic without equality is a weaker version of FOL that has no distinguished equality symbol.

Quantifiers

- The biggest change from propositional logic to first-order logic is the use of quantifiers.
- A **quantifier** is a statement that expresses that some property is true for some or all choices that could be made.
  - Turn predicates into propositions by assigning values to all variables:
    - Predicate P(x): x is even.
    - Proposition P(6): 6 is even.
  - The other way to turn a predicate into a proposition:
    - Add a quantifier like "All" or "Some" that indicates the number of values for which the predicate is true.

A formula that contains variables is not simply true or false unless each of these variables is **bound** by a quantifier

The Universal Quantifier

(∀x)P(x) means that P holds for **all** values of x in the domain associated with that variable
(∀x)P(x): "For all x, x is P"
E.g., (∀x) dolphin(x) → mammal(x)

Definition: The symbol ∀ is the **universal quantifier.**

- The universal quantification of P(x) is the statement P(x) for all values x in the universe, which is written in logical notation as:

  $\forall x\, P(x)$ or sometimes $\forall x \in D, P(x)$.

- A statement of the form $\forall x\, P(x)$ asserts that for every choice of x in our domain, P(x) is true.
  - Example: All professors are people.

    $\forall x\, (Professor(x) \rightarrow People(x))$

The Existential Quantifier

(∃ x)P(x) means that P holds for **some** value of x in the domain associated with that variable
(∃ x)P(x): "For some x, x is P"

- **Definition**: The symbol ∃ is the **existential quantifier.**
  - The existential quantification of P(x) is the statement

    P(x) for some values x in the universe, which is written in logical notation as

    $$\exists x\, P(x).$$

  - A statement of the form ∃x P(x) asserts that for some choice of x in our domain, P(x) is true.
    - Even and prime in the number series.

      ∃x. (Even(x) ∧ Prime(x))

Translating Into First-Order Logic

1. All students are smart.

A universal quantification is a type of quantifier, a logical constant which is interpreted as "given any" or "for all

Incorrect Translation

$\forall x\ (Student(x) \wedge Smart(x))$

This should work for any choice of x, including things that aren't students.

- Although the original statement is true, this logical statement is false. It's therefore not a correct translation.

Correct Translation

$\forall x\ (Student(x) \rightarrow Smart(x))$

Translating Into First-Order Logic

2. There is a student who is smart.

Incorrect Translation

$$\exists x \, (Student(x) \rightarrow Smart(x))$$

- Under an interpretation that the original statement is false; this logical statement is true. It's therefore not a correct translation.

Correct Translation

$$\exists x \, (Student(x) \wedge Smart(x))$$

Translating Into First-Order Logic

- All P's are Q's

  translates as

  $\forall x\,(P(x) \rightarrow Q(x))$

  - $\forall$ quantifier usually is paired with $\rightarrow$

  - In the case of $\forall$, the $\rightarrow$ connective prevents the statement from being false when speaking about some object you don't care about.

Translating Into First-Order Logic

- Some P's are Q's

  translates as

  $\exists x\,(P(x) \wedge Q(x))$

  - $\exists$ quantifier usually is paired with $\wedge$

  - In the case of $\exists$, the $\wedge$ connective prevents the statement from being true when speaking about some object you don't care about.

De Morgan's Laws for Quantifiers

- $\neg\forall x\ P(x) \equiv \exists x\ \neg P(x)$
  - If $\neg\forall x\ P(x)$, then $P(x)$ is not true for every x,
  - For some value a, $P(a)$ is not true. This means that $\neg P(a)$ is true.
  - Since $\neg P(a)$ is true, it is certainly the case that there is some value of x that makes $\neg P(x)$ true.
  - $\exists x\ \neg P(x)$ is true.

- $\neg\exists x\ P(x) \equiv \forall x\ \neg P(x)$

Nesting Quantifiers

- For predicate P(x,y):
  - Switching the order of universal quantifiers *does not* change the meaning

    $\forall x \forall y P(x,y) \leftrightarrow \forall y \forall x P(x,y)$.

  - Similarly, one can switch the order of existential quantifiers

    $\exists x \exists y P(x,y) \leftrightarrow \exists y \exists x P(x,y)$.

- Can not interchange the position of $\forall$ and $\exists$ like this!

Combining Quantifiers

3. Everyone loves someone else

Correct Translation

$\forall x \exists y$ Loves(x,y)

Person(x) : `x' is a Person.

Loves(x,y): `x' loves `y'.

Different from him

$\forall x$ (Person (x) $\rightarrow \exists y$ (Person(y) $\wedge$ x $\neq$ y $\wedge$ Loves(x,y)))

For EVERY person — There is SOMEONE — They LOVE

Combining Quantifiers

4. Someone everyone else loves.

Correct Translation

$\exists x \; \forall y \; Loves(y,x)$

Person(x) : `x' is a Person.

Loves(x,y): `x' loves `y'.

Different from him

$\exists \; x \; (Person \; (x) \wedge \forall \; y \; (Person(y) \wedge x \neq y \rightarrow Loves(y,x)))$

SOMEONE EVERYONE LOVES

Quantifier Ordering

- Order of the quantifiers is important when mixing existential and universal quantifiers!
  - $\forall x \exists y P(x,y)$

    For any choice x, there's some y where P(x, y) is true
  - $\exists x \forall y P(x,y)$

    There is some x where for any choice of y, we get that P(x, y) is true

The inner part has to work for any choice of y, this places a lot of constraints on what x can be.

Negations of Quantified Statements

Negation of a Universal Statement

1. All dogs bark.

Incorrect Negation

No dogs bark.

If at least one dog does not bark, then the original statement is false.

Correct Negation

Some dogs do not bark.

The negation of a universal statement ($\forall x\ \varphi$) is logically equivalent to an existential statement ($\exists x\ \neg\varphi$).

Negations of Quantified Statements

Negation of a Existential Statement

2. Some snowflakes are the same.

Incorrect Negation

Some snowflakes are the different.

Correct Negation

All snowflakes are different.

No snowflakes are the same.

The negation of an existential statement ($\exists x\ \varphi$) is logically equivalent to a universal statement ($\forall x\ \neg\varphi$)

Negations of Quantified Statements

Negation – Pushing the NOT across

3. Everyone loves someone.

$\forall x\ \exists y$ Loves(x,y)

Correct Negation

$\neg\forall x\ \exists y$ Loves(x,y)

$\exists x\ \neg\ \exists y$ Loves(x,y)

$\exists x\ \forall y\ \neg$Loves(x,y)

I could push the not forward by looking at the de-Morganís law that we have little while ago try to argue that for all x Px is same as there exist x not Px.

$\neg\forall x\ P(x) \equiv \exists x\ \neg\ P(x)$

And then I could take this one step forward to replace it with the second de Morganís law that we had seen that not of their exists x P of x is same as for all x not P of x.

$\neg\exists x\ P(x) \equiv \forall x\ \neg\ P(x)$

There is someone who doesn't love anyone.

Negations of Quantified Statements

Negation of a Universal Conditional Statement

Negation of a conditional (if-then) statement is logically equivalent to an AND statement.

$\neg (P \rightarrow Q) \equiv P \wedge \neg Q$

$\neg (\neg P \vee Q)$
$\neg \neg P \wedge \neg Q$
$P \wedge \neg Q$

Negation of a universal statement is logically equivalent to an existential statement.

$\neg \forall x \, \varphi \equiv \exists x \, \neg \varphi$.

Substituting the conditional statement into the universal statement

$$\neg \forall x \, (P(x) \rightarrow Q(x)) \equiv \exists x \, (P(x) \wedge \neg Q(x))$$

Negations of Quantified Statements

Negation of a Universal Conditional Statement

4. If x is a rational number, then $\sqrt{x}$ is a rational number.

   $\forall$x Rational ($\sqrt{x}$ is Rational)

   Correct Negation

   $\exists$x Rational ($\sqrt{x}$ is $\neg$ Rational)

   There exist a rational number x, such that $\sqrt{x}$ is not a rational number.

Distributivity of ∀ over ∧

$$\forall x\, P(x) \wedge \forall x\, Q(x) \equiv \forall x\, (P(x) \wedge Q(x))$$

∀ distributes over ∧

No matter what the domain is, these two propositions always have the same truth value.

This shouldn't be surprising, since for a finite domain, say {1,2,3},

$$\forall x\, P(x) \equiv (P(1) \wedge P(2) \wedge P(3))$$

Further ∧ is commutative and associative, so:

$$\forall x \in \{1, 2, 3\}(P(x) \wedge Q(x))$$

For this example domain

$$\equiv (P(1) \wedge Q(1)) \wedge (P(2) \wedge Q(2)) \wedge (P(3) \wedge Q(3))$$

$$\equiv (P(1) \wedge P(2) \wedge P(3)) \wedge (Q(1) \wedge Q(2) \wedge Q(3))$$

Commutativity/Associativity

$$\equiv \forall x \in \{1, 2, 3\} P(x) \wedge \forall x \in \{1, 2, 3\} Q(x)$$

For this specific example

Though this is only an example domain, the intuition extends to other domains as well, including infinite domains.

Distributing ∃ over ∧

$\exists x\,(P(x) \wedge Q(x)) \not\equiv \exists x\,P(x) \wedge \exists x\,Q(x)$.

The existential quantifier ∃ does not distribute over ∧

Find a counterexample - a universe and predicates P and Q - such that one of the propositions is true and the other is false:

Let U = N.

Set P(x): "x is prime" and Q(x): "x is composite" (i.e. not prime).

$\exists x\,(P(x) \wedge Q(x))$ is False,

$\exists x\,P(x) \wedge \exists x\,Q(x)$ is True.

Distributivity of ∃ over ∨

$\exists x(P(x) \vee Q(x)) \equiv \exists x\, P(x) \vee \exists x\, Q(x)$

∃ distributes over ∨

This rule holds for arbitrary P and Q

Recall: $\forall x\,(P(x) \wedge Q(x)) \equiv \forall x P(x) \wedge \forall x Q(x)$

Replace P by ¬ S and Q by ¬ R.

$\forall x\,(\neg S(x) \wedge \neg R(x)) \equiv \forall x \neg S(x) \wedge \forall x \neg R(x)$

Negate both sides.

$\neg \forall x\,(\neg S(x) \wedge \neg R(x)) \equiv \neg(\forall x \neg S(x) \wedge \forall x \neg R(x))$

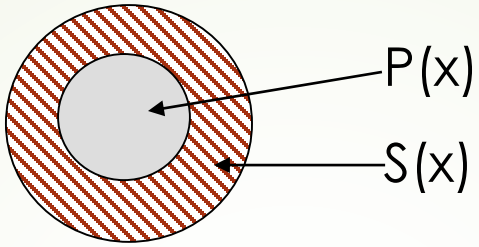
$\exists x \neg(\neg S(x) \wedge \neg R(x)) \equiv \neg \forall x \neg S(x) \vee \neg \forall x \neg R(x))$

$\exists x\,(\neg\neg S(x) \vee \neg\neg R(x)) \equiv \exists x \neg\neg S(x) \vee \exists x \neg\neg R(x))$

✓ $\exists x(S(x) \vee R(x)) \equiv \exists x\, S(x) \vee \exists x\, R(x)$.

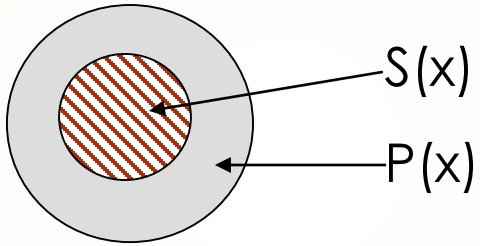
More on definitions

S(x) is a necessary condition of P(x)



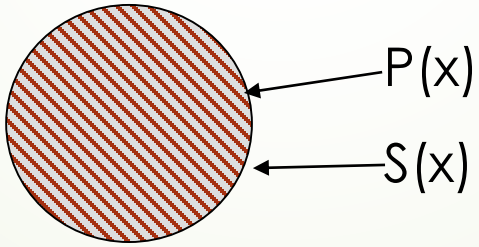
$(\forall x)\ P(x) \Rightarrow S(x)$

S(x) is a sufficient condition of P(x)



$(\forall x)\ P(x) \Leftarrow S(x)$

S(x) is a necessary and sufficient condition of P(x)



$(\forall x)\ P(x) \Leftrightarrow S(x)$